

PetStuff

--- *A Full Stack MERN-Based Multi-Vendor E-Commerce Platform for Pet Owners* ---

Project Type: Full Stack Web Application

Purpose: Learning MERN Stack & Developing My Portfolio

Technology Stack: MongoDB | Express.js | React.js | Node.js

Developer: Mohd Sayam

GitHub Repository: [Repo Link](#)

Video Demonstration: (7–8 min walkthrough – [Add link](#))

1. Executive Summary

PetStuff is a comprehensive **full-stack e-commerce platform** built specifically for pet owners.

The platform bridges the gap between **pet product vendors (store owners)** and **customers**, enabling a smooth and secure shopping experience for pet food, medicines, toys, and accessories.

Developed using the **MERN Stack (MongoDB, Express.js, React.js, Node.js)**, PetStuff emphasizes:

- High performance
- Secure authentication
- Modular architecture
- A modern, responsive user interface

The project demonstrates **real-world full-stack development practices**, including RESTful APIs, role-based access control, cloud media management, and scalable system design.

2. Project Vision & Novelty

Unlike generic e-commerce platforms, PetStuff is designed with a **Multi-Vendor Architecture** at its core.

The platform clearly distinguishes between:

- **Customers** (pet owners)
- **Store Owners (Admins)**

This separation allows multiple vendors to operate independently within a single ecosystem, making the application **scalable and marketplace-ready**.

Key Unique Selling Propositions (USPs)

- **Pet-Centric Categorization**
Products are intelligently categorized by animal type (Dog, Cat, Bird, etc.), enabling faster and more relevant discovery.
- **Dynamic Pricing Engine**
Sale prices and discount percentages are automatically calculated, ensuring consistency and reducing manual errors.

- **Hybrid Authentication System**

Users can authenticate via:

- Email & Password
- Google OAuth (one-click login)

- **Vendor Autonomy**

Each store owner gets a **dedicated dashboard** to manage products, orders, and analytics without affecting other vendors or global platform settings.

3. Technology Stack

3.1 Frontend (Client-Side)

- **Framework:** React.js (Vite)

Chosen for fast development builds and optimized performance.

- **Styling:** Tailwind CSS

Enables a mobile-first, responsive, and consistent UI design.

- **State Management:** React Context API

Used for managing global states such as:

- Authentication
- Shopping Cart
- UI Theme

- **Network Layer:** Axios

Configured with interceptors for secure token-based API communication.

- **UX Enhancements:**

- `react-hot-toast` for real-time user feedback
 - `lucide-react` for clean and modern iconography
-

3.2 Backend (Server-Side)

- **Runtime & Framework:** Node.js with Express.js

Provides scalable and maintainable RESTful API services.

- **Database:** MongoDB (NoSQL)

Enables flexible schema design and high-performance queries.

- **Authentication & Security:**

- JSON Web Tokens (JWT) for session handling
- Passport.js for Google OAuth authentication strategies

- **Media Management:**

- Cloudinary for optimized image storage and delivery

- **Email Communication:**
 - Nodemailer for transactional emails such as:
 - Email verification
 - Password reset flows
-

4. Detailed Features

4.1 Customer Features

- **Seamless Onboarding**
Simple sign-up process with support for Google OAuth.
 - **Smart Search & Product Discovery**
Filter products by:
 - Animal type (Dog, Cat, etc.)
 - Product category (Food, Toys, Medicines)
 - **Shopping Cart System**
Persistent cart allowing:
 - Quantity updates
 - Real-time price calculation
 - **Secure Checkout**
Order placement with address management and order summary.
 - **Order Tracking**
Real-time order status updates:
Processing → Shipped → Delivered
 - **Profile Management**
Secure handling of personal details and password recovery.
-

4.2 Store Owner (Admin) Features

- **Store Creation**
Automated setup of a digital storefront with location and descriptive metadata.
- **Product Lifecycle Management**
Full **CRUD operations**:
 - Add new products
 - Update pricing and stock
 - Delete discontinued items
- **Order Fulfillment System**
Store owners can view and manage only the orders related to their products.

- **Analytics Dashboard**
Sales and order insights visualized using **Recharts**, enabling data-driven decisions.

5. Database Architecture & Entity Relationships

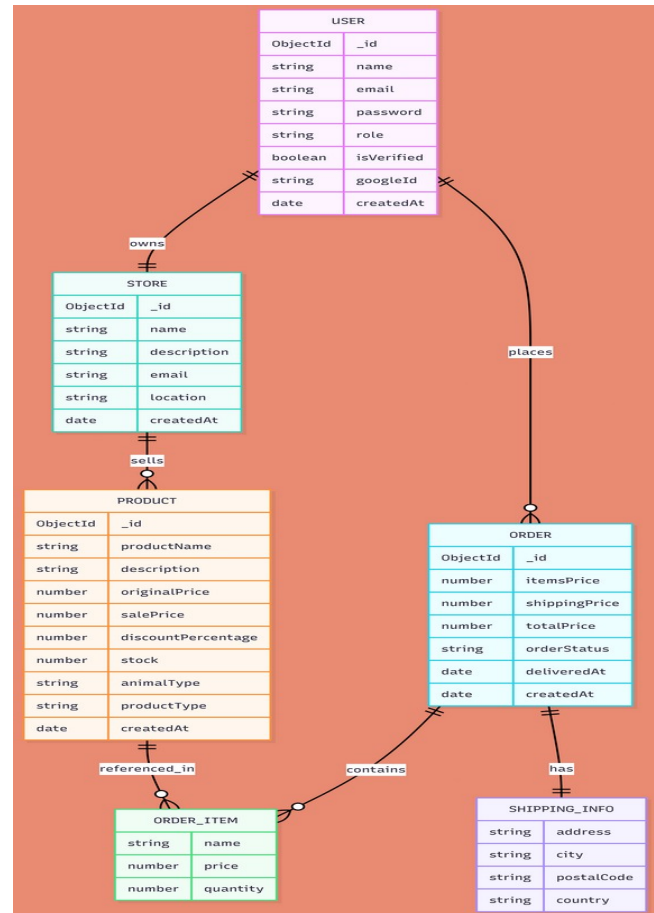
Description:

The PetStuff platform uses a structured MongoDB schema design to maintain relational integrity across users, stores, products, and orders.

The database follows a normalized approach where:

- Users can act as Customers or Store Owners
- Each Store is owned by a single Admin
- Products belong to specific Stores
- Orders maintain immutable product snapshots to ensure historical accuracy

The following ER diagram represents the complete database structure and relationships used in the system.



6. Functional Workflows & Highlights

- **Authentication Flow**
Email verification via tokenized links or instant login via Google OAuth.
- **Purchase Flow**
Product added → Checkout → Stock deduction → Order creation → Store owner notification.
- **Deployment Strategy**
Implements a “**Chicken & Egg**” deployment pattern on **Vercel** to resolve circular environment variable dependencies between frontend and backend securely.

7. Future Roadmap

- Integration of **Stripe** / **Razorpay** payment gateways
- Product **review and rating system**
- **Real-time chat** between store owners and customers
- Advanced analytics and performance optimizations
- Mobile application version

8. Video Demonstration

A **7–8 minute detailed walkthrough video** is attached with this documentation, explaining:

- Project motivation
- UI walkthrough
- Authentication flow
- Product & order management
- Backend architecture and APIs

The video is designed to be **concise, engaging, and well explained**.

Video Link: [Video](#)